Energy-Planner: Intelligent Energy Scheduling Assistant

Gerardo Lauro
Francesco Simone Pirozzi
Alessandro Lavello

October 14, 2025

Contents

1 Introduction **4**

1.1 Why this matters . . . 4

1.2 Energy-Planner at a glance . . . 4

1.3 This document . . . 5

2 System Architecture **6**

2.1 Design goals . . . 6

2.2 High-level data flow . . . 6

2.3 Modules . . . 7

2.3.1 Conversational front-end (LLM Orchestrator) . . . 7

2.3.2 Parser & Triple Store . . . 7

2.3.3 Normalization . . . 8

2.3.4 Validation Engine . . . 8

2.3.5 Price Service . . . 8

2.3.6 Instance Compiler . . . 8

2.3.7 Optimization Layer . . . 9

2.3.8 Reporting & Recommendations . . . 9

2.4 JSON schema (abridged) . . . 9

2.5 Configuration and deployment . . . 10

2.6 Scalability and extensibility . . . 10

2.7 Assumptions and limitations . . . 11

3 Conversation-to-JSON Pipeline **12**

3.1 Overview . . . 12

3.2 Stage 1: Conversational extraction . . . 12

3.2.1 Interaction principles . . . 12

3.2.2 Structured saves . . . 13

3.3 Stage 2: Parsing and graph construction . . . 13

3.3.1 Example graph . . . 13

3.4 Stage 3: Normalization . . . 14

3.4.1 Temporal grid . . . 14

3.4.2 Unit harmonization . . . 14
3.4.3 Assumption tagging . . . 14
3.5 Stage 4: Validation . . . 14
3.6 Stage 5: Compilation to JSON . . . 15
3.7 Example dialogue-to-instance . . . 15
3.7.1 Input conversation . . . 15
3.7.2 Output instance (simplified) . . . 16
3.8 Traceability and reproducibility . . . 16
3.9 Summary . . . 17

4 Price Forecasting Integration **18**
4.1 Purpose . . . 18
4.2 Integration architecture . . . 18
4.3 Forecast models and data sources . . . 19
4.4 Usage within optimization . . . 19
4.5 Summary . . . 19

5 Optimization Module **20**
5.1 Objective and scope . . . 20
5.2 Heuristic scheduling (implemented) . . . 20
5.2.1 Overview . . . 20
5.2.2 Inputs and outputs . . . 21
5.3 Mathematical Programming (MIP) formulation (implemented) . . . 21
5.3.1 Model structure . . . 21
5.4 Comparison and scalability . . . 22
5.5 Integration with price forecasts . . . 22
5.6 Summary . . . 22

6 Evaluation Scenarios **23**
6.1 Testing methodology . . . 23
6.2 Scenario 1: Variable tariff with hourly prices (Milan) . . . 23
6.2.1 Setup . . . 23
6.2.2 Outcome . . . 24
6.3 Scenario 2: Fixed tariff with contractual power limit (Rome) . . . 24
6.3.1 Setup . . . 24
6.3.2 Outcome . . . 24
6.4 Scenario 3: Missing data fallback (Naples) . . . 25
6.4.1 Setup . . . 25
6.4.2 Outcome . . . 25
6.5 Scenario 4: Multi-activity with overlapping resources . . . 25

6.5.1 Setup . . . 25
6.5.2 Outcome . . . 25
6.6 Discussion . . . 26
6.7 Summary . . . 26

7 Scalability and Deployment **27**
7.1 Dimensions of scalability . . . 27
7.2 Functional scalability . . . 27
7.3 Computational scalability . . . 28
7.4 Deployment architecture . . . 28
7.4.1 Prototype stack . . . 28
7.4.2 Production-ready deployment . . . 28
7.5 Security and reliability . . . 29
7.6 Cloud/on-prem integration . . . 29
7.7 Future scalability roadmap . . . 29
7.8 Summary . . . 29

8 User Experience and Roadmap **31**
8.1 Current UX state . . . 31
8.2 Planned improvements . . . 31
8.2.1 Short term (prototype to beta) . . . 31
8.2.2 Medium term (beta to production) . . . 32
8.2.3 Long term (scaling and integration) . . . 32
8.3 User testing feedback . . . 32
8.4 UX roadmap summary . . . 33
8.5 Summary . . . 33

9 Conclusions **34**
9.1 Achievements . . . 34
9.2 Key outcomes . . . 34
9.3 Alignment with Enel Innothon goals . . . 35
9.4 Future work . . . 35
9.5 Final remarks . . . 36

Chapter 1

Introduction

1.1 Why this matters

Electricity is a major lever for SME margins, yet planning is often reactive and decoupled from real drivers: hourly prices, machine flexibility, workforce shifts, and seasonal patterns. Price volatility and electrification raise both risk and opportunity. What SMEs need is not another dashboard, but a compact way to translate "how we work in a typical day" into energy-aware schedules.

The gap. Missed shifting of discretionary loads to cheap hours; unmanaged peaks vs contractual limits; little use of price and weather signals; scarce feedback on what actually saved money.

Design requirement. A tool must understand natural language, formalize constraints, and return a feasible plan that reduces cost and/or peak without breaking operations—while staying usable under incomplete inputs.

1.2 Energy-Planner at a glance

What it is. A conversational planner that turns a short interview into an optimization-ready instance and a schedule.
How it works. *chat* $\rightarrow$ *validated JSON* $\rightarrow$ *optimization* (heuristic and MIP, both implemented) $\rightarrow$ *recommendations*.
What it uses. Day-ahead hourly price integration is implemented and upsampled to the 5-minute grid to anticipate expensive periods and shift discretionary loads.

Key features.

- **Adaptive objective:** `min_cost` for variable/indexed tariffs; `min_peak` for flat tariffs (optional peak caps).
- **Robust defaults:** graceful fallbacks when data is missing; all assumptions are logged for audit.
- **Solver-ready:** clean JSON schema (time horizon, pricing vector, machines, workforce, activities, constraints).
- **Dual solver backends:** heuristic and MIP are implemented and selectable per instance.
- **Scalable:** modular components, cloud/on-prem ready, multi-city feeds, multiple solvers.

1.3 This document

Chapter 2 sketches the architecture; Chapter 3 details the conversation-to-JSON pipeline; Chapter 4 covers price forecasting and integration; Chapter 5 discusses heuristic vs MIP scheduling. Later chapters include scenarios, scalability & deployment, UX roadmap, and conclusions.

Chapter 2

System Architecture

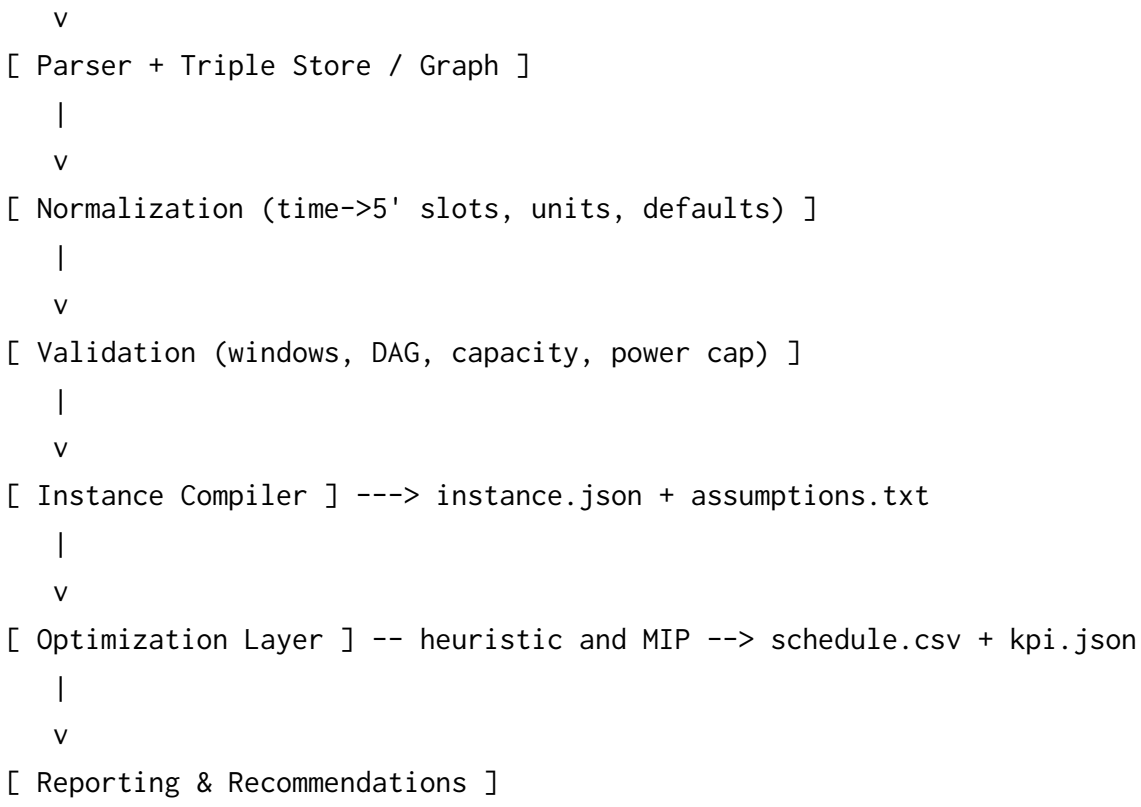
This chapter describes the end-to-end architecture of *Energy-Planner*, from the conversational front-end to the solver-ready JSON and recommendations. The architecture is modular, provider-agnostic for LLMs, and designed to operate with partial inputs while maintaining auditability, with implemented price integration and solver backends.

2.1 Design goals

- **Separation of concerns**: conversation, parsing/validation, price service, instance compilation, optimization, and reporting are isolated modules.
- **Deterministic hand-offs**: each stage produces a well-defined artifact (e.g., triples, normalized graph, JSON instance, assumptions log).
- **Resilience**: defaults and fallbacks keep the pipeline usable with incomplete data; every assumption is logged.
- **Scalability**: swappable price connectors, local/cloud LLMs, and multiple solvers (heuristic/MIP).
- **Auditability**: reproducible runs via versioned JSON and explicit reports of assumptions and validations.

2.2 High-level data flow

```
User (plain language)
   |
   v
[ Conversational Front-end (LLM Orchestrator) ]
   |  >>> SAVE [entity, attribute, value]
```



2.3 Modules

2.3.1 Conversational front-end (LLM Orchestrator)

- Presents a short, guided interview (1–2 targeted questions per turn) to elicit: city/region, tariff type, workforce/shift, machines (kW), flexible activities (durations, windows, resources), precedences, and optional power cap.
- Enforces guardrails: first turn must be a question, structured saves via `>>> SAVE` blocks, automatic retries on malformed saves, no leakage of internal prompts.
- Model-agnostic: runs with local models (e.g., via Ollama) or cloud APIs; behavior is identical thanks to a thin provider interface.

2.3.2 Parser & Triple Store

- Extracts and validates `>>> SAVE` blocks formatted as JSON triples `[subject, attribute, value]`.
- Builds an in-memory graph (entity nodes for `price`, `machine:*`, `activity:*`, `workforce`; edges for requirements and precedences).
- Rejects incomplete or inconsistent triples; assigns issue codes for later reporting.

2.3.3 Normalization

- Discretizes time to a 5-minute slot grid (e.g., 24h horizon = 288 slots). Rounds durations and windows to nearest slot boundaries.
- Harmonizes units (kW, minutes) and attaches confidence/assumption flags to derived values.
- Applies robust defaults when unspecified: workforce capacity (`1` across slots), flat price vector, or placeholder machine power (flagged for review).

2.3.4 Validation Engine

- **Window feasibility**: each activity's duration must fit into its available window.
- **Precedence DAG**: precedence constraints must be acyclic.
- **Capacity**: workforce per-slot load $\leq$ available capacity; machines cannot run two activities concurrently.
- **Electrical constraints**: optional contractual power cap must not be exceeded by concurrent machine loads.
- Emits warnings/errors with actionable messages; blocks compilation on hard infeasibilities.

2.3.5 Price Service

- Provides (implemented) day-ahead hourly prices for supported cities/areas; when available, integrates external forecasts.
- Upsamples hourly values to the 5-minute slot resolution (12 slots per hour).
- Fallback (implemented): if prices are missing, uses a documented flat rate for the horizon and records the assumption.

2.3.6 Instance Compiler

Produces a solver-ready JSON with the following sections:

- **time**: horizon length, slot size (5 minutes), and optional start timestamp.
- **pricing**: vector `p[0..H-1]`, one value per slot.
- **workforce**: per-slot capacity (default `1` if unspecified).

- **machines**: list of machines with `{id, name, power_kw}`.
- **activities**: list with `{id, name, duration_slots, window_start, window_end, requires_machi`
- **objective**: `min_cost` (variable/indexed tariffs) or `min_peak` (flat tariffs); optional peak cap.
- **assumptions**: structured list of defaults and inferred values with reasons.

Artifacts are persisted as `out/instance.json` and `out/assumptions.txt`.

2.3.7 Optimization Layer

- **Heuristic engine**: constructs a feasible schedule, then improves it via local search (e.g., randomized greedy + swaps), honoring all constraints.
- **MIP interface**: implemented with an exact solver backend (e.g., OR-Tools/PuLP); discussed in detail in Chapter 5.
- Objective mapping is automatic from tariff type; mixed objectives (cost with peak cap) are supported.

2.3.8 Reporting & Recommendations

- Summarizes the plan (start times, overlaps avoided, expected cost/peak metrics) and highlights the savings drivers.
- Exposes all warnings and assumptions; invites user corrections for low-confidence inputs.
- Exports machine-readable outputs (JSON/CSV) for downstream dashboards.
- Exports `schedule.csv` (start,end,machine,activity,workers) and `kpi.json` (cost_eur, peak_kw, shifted_pct, feasibility).

2.4 JSON schema (abridged)

```
{
  "time": { "slot_minutes": 5, "horizon_slots": 288 },
  "pricing": [0.27, 0.27, 0.27, 0.19],
  "workforce": { "capacity": [1, 1, 1, 1] },
  "machines": [ {"id":"M1","name":"Oven","power_kw":4.0} ],
  "activities": [
    {
```

```
      "id": "A1",
      "name": "Baking",
      "duration_slots": 9,
      "window_start": 108,
      "window_end": 144,
      "requires_machine": "M1",
      "workers": 1,
      "precedence": ["A0"]
    }
  ],
  "objective": "min_cost",
  "peak_cap_kw": 6.0,
  "assumptions": [
    {"code":"PRICE_FLAT","detail":"Using flat 0.25 EUR/kWh due to
       missing feed"}
  ]
}
```


2.5 Configuration and deployment

- **LLM backends**: selectable at startup (local vs cloud). API keys and model names supplied via environment variables or a config file.
- **Persistence**: file-based by default (`out/*`). Optional DB adapters (NoSQL/SQL) can be added without altering the pipeline.
- **Stateless services**: each stage is stateless w.r.t. optimization runs; conversation state is kept in-session and exported via artifacts.
- **Logging**: structured logs for saves, validation issues, and assumptions support reproducibility.

2.6 Scalability and extensibility

- **Multi-city feeds**: price connectors are pluggable; adding cities or sources requires only a new adapter.
- **Richer constraints**: extend the JSON to multiple teams, skill matrices, maintenance windows, or machine changeover times.
- **Solver portfolio**: choose heuristic for speed or MIP for optimality; add metaheuristics or hybrid warm-starts.

- **Interfaces**: swap CLI with a web/mobile UI; the backend contracts remain stable.

2.7 Assumptions and limitations

- 5-minute discretization is a design choice balancing fidelity and tractability; it can be tuned per deployment.
- Price integration currently assumes hourly feeds; intra-hour prices are represented by replication per slot.
- In the absence of user-supplied workforce schedules, capacity defaults to 1 and is explicitly flagged in the report.

Chapter 3

Conversation-to-JSON Pipeline

This chapter describes how natural-language dialogue between the user and the assistant is transformed into a rigorous, validated JSON instance suitable for optimization. The process combines language understanding (LLM), deterministic parsing, data normalization, and constraint validation to guarantee both usability and correctness.

3.1 Overview

The *conversation-to-JSON pipeline* converts unstructured information about a company's daily operations into a formal scheduling instance. The user only describes their "typical day": machines, shifts, tasks, and any constraints. From this dialogue, the system extracts structured triples, builds an internal graph of relations, normalizes times and units, validates constraints, and finally compiles an `instance.json` file.

`Plain Language → Triples → Graph → Normalization → Validation → JSON Instance`

Each stage is independent and produces explicit logs, ensuring full traceability and easy debugging.

3.2 Stage 1: Conversational extraction

3.2.1 Interaction principles

The assistant conducts a guided dialogue with strict policies:

- Asks at most two targeted questions per turn.
- Reformulates each confirmed answer into structured data.
- Avoids technical jargon; uses natural Italian or English phrasing.

- Triggers a structured save whenever a relevant fact is confirmed.
- Structured saves via `>>> SAVE` blocks are implemented; malformed saves trigger automatic retries.

3.2.2 Structured saves

When the user provides a relevant piece of information, the assistant emits a block:

```
>>> SAVE ["activity:A1","duration_min",45]
>>> SAVE ["activity:A1","requires_machine","M1"]
>>> SAVE ["machine:M1","power_kw",4.0]
```

Each line is a JSON array of the form `[subject, attribute, value]`. Only such lines are captured by the parser; everything else is ignored. This strict syntax ensures that every saved element is machine-readable and traceable.

3.3 Stage 2: Parsing and graph construction

The parser monitors the assistant output stream and collects all valid `>>> SAVE` blocks. Each triple becomes a node or an edge in the internal data graph:

- **Entities**: machines, activities, workforce, pricing, tariff, constraints.
- **Attributes**: numeric (duration, kW, hours) or categorical (tariff type, precedence link).
- **Relations**: activities require certain machines, depend on other activities, or share resources.

If any triple is malformed or inconsistent (e.g., missing value, wrong type), the parser discards it and issues a correction request. All accepted triples are timestamped and versioned.

3.3.1 Example graph

```
activity:Mixing --requires_machine--> machine:M1
activity:Baking --after--> activity:Mixing
machine:M1 --power_kw--> 3.5
workforce --available_slots--> [0..192]
```

This graph serves as the intermediate knowledge base for subsequent steps.

3.4 Stage 3: Normalization

3.4.1 Temporal grid

All time-related data are discretized onto a 5-minute slot grid (default horizon: 24 hours = 288 slots). Durations and time windows are rounded to the nearest slot boundary. Slot index mapping (00:00→0, 00:05→1, . . . , 09:00→108, 12:00→144) is fixed by design.

For example, an activity described as "30 minutes between 9:00 and 12:00" becomes:

```
window_start = 108
window_end   = 144
slots_needed = 6
```

3.4.2 Unit harmonization

Units are converted to consistent bases (kW for power, minutes for duration). Textual durations ("half an hour") are parsed via simple linguistic patterns. Any missing or ambiguous values are assigned defaults with confidence flags (e.g., $\mathrm{LOW}_{C}ONF, ASSUMED_{V}ALUE$).

3.4.3 Assumption tagging

Every default or inference is logged in an internal structure like:

```
{"assumption": "WORKFORCE_DEFAULT", "detail": "Set workforce=1 per slot"}
```

These entries appear later in `assumptions.txt` for transparency.

3.5 Stage 4: Validation

Before compilation, the system validates all structural and logical constraints:

- **Window feasibility**: each activity fits inside its declared time window.
- **Precedence DAG**: all dependencies form an acyclic graph.
- **Resource limits**: sum of active tasks per slot workforce capacity.
- **Machine exclusivity**: each machine can execute only one task at a time.
- **Power cap**: concurrent machine power contractual limit.

Policy hard vs soft. *Hard* (blocking): precedence cycles, infeasible windows, machine overlap, workforce overflow, power cap exceedance.
Soft (with fallback): missing prices (flat placeholder), unspecified workforce (default 1). All soft cases are logged in `assumptions.txt`.

Violations trigger friendly warnings during the conversation so the user can adjust inputs immediately. Hard infeasibilities stop compilation until fixed.

3.6 Stage 5: Compilation to JSON

Once data are consistent, the compiler exports the following structure:

```
{
  "time": {"slot_minutes": 5, "horizon_slots": 288},
  "pricing": [0.25, 0.25, 0.25],
  "workforce": {"capacity": [1, 1, 1]},
  "machines": [{"id":"M1","power_kw":4.0}],
  "activities": [
    {"id":"A1",
     "duration_slots":6,
     "window_start":108,
     "window_end":144,
     "requires_machine":"M1",
     "workers":1,
     "precedence":["A0"]}
  ],
  "objective":"min_peak",
  "assumptions":[
    {"code":"PRICE_FLAT","detail":"Using flat 0.25 EUR/kWh due to
       missing feed"}
  ]
}
```

Each JSON file is deterministic: given the same inputs, the same structure is produced. This allows reproducibility and downstream integration with solvers.

3.7 Example dialogue-to-instance

3.7.1 Input conversation

> **User:** We are in Rome with a fixed tariff and a 6 kW contract. Our oven is 4 kW and mixer 3 kW. Mixing (30 min) must happen between 9 and 12, baking

(45 min) after mixing.

Assistant: OK, I recorded city=Rome, offer=FIX, contract limit=6 kW...

»> SAVE ["price","city","Rome"]

»> SAVE ["offer","type","FIX"]

»> SAVE ["constraint","max$_p$ower$_k$w", 6]

$>>> SAVE[\text{"}machine : M1\text{"}, \text{"}name\text{"}, \text{"}Mixer\text{"}]$

$>>> SAVE[\text{"}machine : M1\text{"}, \text{"}power_kw\text{"}, 3]$

$>>> SAVE[\text{"}machine : M2\text{"}, \text{"}name\text{"}, \text{"}Oven\text{"}]$

$>>> SAVE[\text{"}machine : M2\text{"}, \text{"}power_kw\text{"}, 4]$

$>>> SAVE[\text{"}activity : A1\text{"}, \text{"}name\text{"}, \text{"}Mixing\text{"}]$

$>>> SAVE[\text{"}activity : A1\text{"}, \text{"}duration_min\text{"}, 30]$

$>>> SAVE[\text{"}activity : A1\text{"}, \text{"}window_start\text{"}, \text{"}09 : 00\text{"}]$

$>>> SAVE[\text{"}activity : A1\text{"}, \text{"}window_end\text{"}, \text{"}12 : 00\text{"}]$

$>>> SAVE[\text{"}activity : A2\text{"}, \text{"}name\text{"}, \text{"}Baking\text{"}]$

$>>> SAVE[\text{"}activity : A2\text{"}, \text{"}duration_min\text{"}, 45]$

$>>> SAVE[\text{"}activity : A2\text{"}, \text{"}window_start\text{"}, \text{"}09 : 00\text{"}]$

$>>> SAVE[\text{"}activity : A2\text{"}, \text{"}window_end\text{"}, \text{"}12 : 00\text{"}]$

$>>> SAVE[\text{"}activity : A2\text{"}, \text{"}precedence\text{"}, \text{"}A1\text{"}]$

3.7.2 Output instance (simplified)

```
objective = min_peak
max_power = 6 kW
schedule suggestion:
  Mix A1 at 09:00 → 09:30
  Bake A2 at 09:30 → 10:15
peak power = 4 kW (within limit)
```

3.8 Traceability and reproducibility

Every session outputs two synchronized files:

- **instance.json** – full structured data for solvers.
- **assumptions.txt** – human-readable log of defaults, warnings, and inferred data.

This dual output ensures the same scenario can be reloaded or verified independently of the conversation, satisfying auditability and transparency requirements.

3.9 Summary

The conversation-to-JSON pipeline transforms qualitative input into a formal optimization instance with guaranteed structure and consistency. This mechanism bridges user-friendly interaction and rigorous energy planning, fulfilling the Innothon goal of making advanced optimization accessible to SME owners.

Chapter 4

Price Forecasting Integration

This chapter explains how hourly electricity price forecasts are integrated within the Energy-Planner optimization pipeline. The purpose is not to replicate the full technical details—these are provided in the dedicated document *Energy Price Forecaster – Technical Documentation*—but to clarify how predicted prices interact with scheduling and optimization.

4.1 Purpose

Accurate price forecasting enables the system to anticipate expensive hours and shift flexible loads accordingly. The forecasts serve as the dynamic cost vector in the optimization problem, replacing static or user-supplied tariffs. This transforms the planner from a reactive to a proactive decision-support tool.

4.2 Integration architecture

- The price forecaster module **is implemented** and provides hourly predictions for each supported region (e.g., Rome, Milan, Naples).
- These hourly values are upsampled to match the 5-minute slot resolution of the scheduling engine (each hourly price repeated 12 times).
- The resulting array becomes the `pricing[]` vector inside `instance.json`.
- When live data are unavailable, the planner **applies an implemented** flat fallback rate (default 0.25 €/kWh) and records this assumption in `assumptions.txt`.

4.3 Forecast models and data sources

The forecasting component described in the companion document is **implemented** using a hybrid time-series and machine-learning pipeline. Typical inputs include historical day-ahead market prices, weather signals, and calendar features; model families can include tree-based learners and temporal deep nets. Full methodology, training routine and feature engineering are provided in the companion *Energy_Price_Forecaster_Technical_Documentation.pdf*.

4.4 Usage within optimization

Depending on the contract type:

- For variable/indexed tariffs, the objective is `min_cost`, and the predicted prices directly drive scheduling.
- For flat tariffs, price forecasts are stored for information only; optimization instead focuses on `min_peak`.

Each optimization run therefore adapts its objective automatically to the data context. This coupling is **implemented** end-to-end within the pipeline.

4.5 Summary

The price forecasting module supplies the temporal cost structure that makes Energy-Planner anticipatory rather than static. Forecasts are seamlessly injected into the optimization schema, while all technical and methodological specifics—model selection, retraining strategy, and validation procedures—are documented in the external reference file **Energy_Price_Forecaster_Technical_Documentation.pdf**.

Chapter 5

Optimization Module

This chapter describes the optimization component responsible for generating the final energy-aware schedule. It builds upon the structured instance produced by the conversation-to-JSON pipeline and leverages either heuristic or mathematical programming (MIP) techniques to minimize energy cost or power peak. Full implementation details are available in the companion documentation *Energy_Optimization_module_technical_implementation.pdf*.

5.1 Objective and scope

The optimization layer transforms the validated instance into a concrete schedule that respects all operational constraints while minimizing one of two objectives:

- **Cost minimization (`min_cost`)** – for variable or indexed tariffs, the system schedules activities during the cheapest forecasted slots.
- **Peak minimization (`min_peak`)** – for flat-rate tariffs, it reduces the maximum concurrent power draw to stay within contractual limits.

The module **automatically selects the objective** based on the tariff type identified earlier in the pipeline. Mixed configurations (e.g., `min_cost` with an explicit peak cap) are supported.

5.2 Heuristic scheduling (implemented)

5.2.1 Overview

A lightweight heuristic scheduler is **implemented** for near-real-time planning with minimal computation. The approach combines constructive and improvement phases:

1. **Feasible initialization:** activities are sorted by priority (e.g., window tightness and power) and placed into available slots without violating precedence or resource constraints.

2. **Local refinement:** a randomized greedy procedure explores alternative start times to reduce cost or flatten peak load.

3. **Metaheuristic enhancement:** swap/shift moves (simulated-annealing style) diversify and refine the plan.

All constraints from the instance—precedence DAG, workforce limits, machine exclusivity, and power cap—are enforced at each step.

5.2.2 Inputs and outputs

- **Input:** `instance.json` (time, pricing vector, workforce, machines, activities, objective).

- **Output:** `schedule.csv` (start, end, machine, activity, workers) and `kpi.json` (cost_eur, peak_kw, feasibility flags).

5.3 Mathematical Programming (MIP) formulation (implemented)

5.3.1 Model structure

A mixed-integer programming formulation is **implemented** in line with the JSON schema. One common encoding is:

Decision variables: $\mathrm{x}_{a,t} = \{\, 1\ if activity \mathrm{a} starts at time slot \mathrm{t},$
$0 otherwise$
$\mathrm{y}_{a,s} = \{\, 1\ if activity \mathrm{a} is active in slot \mathrm{s},$
$0 otherwise$

Constraints:

- Single start within window for each activity.

- Duration consistency: consecutive active slots after start.

- Workforce capacity respected per slot.

- Machine non-overlap on shared resources.

- Optional contractual power cap per slot.

- Precedence: if $a_1 \prec a_2$, then $end(a_1) < start(a_2)$.

Objective functions:

- **`min_cost`:** $\min \sum_s price_s \cdot \sum_a y_{a,s} \cdot power_a$
- **`min_peak`:** $\min Z$ with $Z \geq \sum_a y_{a,s} \cdot power_a$ for all s.

The implementation supports running the heuristic or the MIP via configuration and allows using a heuristic schedule as a warm start for the MIP.

5.4 Comparison and scalability

- **Heuristic mode:** fast and suitable for frequent replanning or interactive use.
- **MIP mode:** exact and suitable for benchmarking and high-stakes plans.
- **Hybrid approach:** heuristic-generated schedules can warm-start the MIP solver.
- **Parallelization:** independent heuristic runs with different seeds are supported.

5.5 Integration with price forecasts

Price forecasts provide the time-dependent coefficients in the `min_cost` objective. When forecasts indicate higher prices in certain intervals, the optimization engine naturally shifts flexible loads toward lower-cost windows while maintaining feasibility. This coupling is **implemented** within the pipeline.

5.6 Summary

The optimization module bridges descriptive data and actionable scheduling through two complementary approaches—heuristic and MIP—both **implemented** and driven by the same standardized instance schema and forecast inputs. Detailed mathematical and engineering notes are available in the companion documentation.

Chapter 6

Evaluation Scenarios

This chapter presents a set of practical evaluation scenarios used to test and validate the Energy-Planner prototype. Each scenario reproduces a realistic SME configuration with varying tariff types, machine setups, and constraints. The purpose is to demonstrate correctness, robustness, and adaptability of the conversation-to-optimization pipeline.

6.1 Testing methodology

The evaluation focused on three complementary dimensions:

- **Functional correctness:** verifying that all inputs provided during the conversation are correctly reflected in the generated `instance.json`.
- **Optimization consistency:** confirming that both heuristic and MIP solvers yield feasible, coherent schedules under the given constraints.
- **Usability and robustness:** assessing whether the system handles incomplete or ambiguous user inputs gracefully through defaults and logged assumptions.

Each scenario follows the same structure: conversational setup, generated instance summary, and resulting schedule (cost or peak metrics).

6.2 Scenario 1: Variable tariff with hourly prices (Milan)

6.2.1 Setup

- City: Milan, variable PUN-indexed tariff.
- Two machines: Mixer (3 kW), Oven (4 kW).

- Activities: Mixing (30 min), Baking (45 min, after Mixing).
- Workforce: 1 operator available 08:00–16:00.
- Power cap: none.

6.2.2 Outcome

- Objective: `min_cost`.
- The optimization shifts flexible tasks into cheaper forecasted bands, producing a feasible schedule that reflects the price signal.

This case demonstrates correct forecast integration and sensitivity to dynamic hourly prices.

6.3 Scenario 2: Fixed tariff with contractual power limit (Rome)

6.3.1 Setup

- City: Rome, fixed tariff (flat 0.25 €/kWh).
- Contractual power limit: 6 kW.
- Machines: Mixer (3 kW), Oven (4 kW).
- Activities: Mixing (30 min), Baking (45 min, after Mixing).
- Workforce: 1 operator (08:00–16:00).

6.3.2 Outcome

- Objective: `min_peak`.
- The optimization staggers tasks to respect the 6 kW limit, maintaining feasibility and avoiding overlaps on shared resources.

This scenario validates constraint enforcement and peak minimization logic.

6.4 Scenario 3: Missing data fallback (Naples)

6.4.1 Setup

- City: Naples, variable tariff but no price data available.
- Machines: Compressor (5 kW), Cooler (2 kW).
- Activities: Air Compression (20 min), Cooling (40 min, after Compression).
- Workforce: unspecified (defaulted to 1).

6.4.2 Outcome

- Forecast unavailable → applied flat fallback (documented in `assumptions.txt`).
- Objective defaults to `min_cost` with uniform pricing.
- The schedule remains feasible; the user is prompted to update price data when available.

This demonstrates resilience under partial information and transparent logging of inferred values.

6.5 Scenario 4: Multi-activity with overlapping resources

6.5.1 Setup

- Four flexible activities (A–D) across two machines.
- Workforce capacity: 2 operators.
- Non-uniform cost curve with morning/evening bands.

6.5.2 Outcome

- Heuristic mode produces a feasible plan, honoring precedences and capacities.
- MIP mode confirms the plan's quality under the same constraints.
- Peak load is reduced compared to naive concurrent execution.

This case shows scalability with more tasks and verifies solver consistency.

6.6 Discussion

Across all tests, Energy-Planner produces valid, explainable, and energy-aware schedules. The modularity of the architecture enables smooth switching between heuristic and MIP solvers. Usability checks confirm that even incomplete conversations lead to usable results, with clear indications of assumptions and warnings. The system thus satisfies both the functional and experiential goals outlined in the Enel Innothon challenge.

6.7 Summary

The evaluation shows that the Energy-Planner prototype is functionally robust, efficient in practice, and ready for extension. It successfully bridges conversational input, predictive pricing, and optimized scheduling across diverse real-world conditions.

Chapter 7

Scalability and Deployment

This chapter outlines how the Energy-Planner architecture can evolve from prototype to production, focusing on scalability in functionality, performance, and deployment flexibility. The design enables seamless growth—supporting multiple users, richer data, and real-time integration—without altering the system's conceptual foundation.

7.1 Dimensions of scalability

Energy-Planner is scalable along three main axes:

1. **Functional scalability**: adding new modules or extending the JSON schema to include richer constraints, additional tariff types, or new cities.
2. **Computational scalability**: handling larger problem sizes (more activities, longer horizons) and concurrent optimization sessions.
3. **Operational scalability**: deploying the system in diverse environments (local, cloud, or hybrid) while maintaining performance and data integrity.

7.2 Functional scalability

- **New data sources**: integrating additional pricing APIs or renewable generation forecasts (e.g., solar PV, weather-based demand prediction).
- **Extended constraints**: including workforce shifts, maintenance windows, machine changeover times, or skill-based task allocation.
- **Dynamic adaptation**: continuous re-optimization as new price forecasts or consumption data arrive.
- **Multi-day horizons**: expanding beyond the current 24-hour scope to weekly planning using rolling horizon logic.

7.3 Computational scalability

The platform supports multiple solver backends and distributed workloads; **heuristic and MIP modes are implemented**.

- **Heuristic mode:** practical linear scaling with the number of activities, suitable for quick decisions or embedded devices.
- **MIP mode:** can leverage parallel/cloud compute (e.g., solver farms) when needed.
- **Hybrid warm-starts:** heuristic results can initialize MIP solvers to reduce time-to-solution.
- **Decoupled processing:** conversation parsing and optimization can be separated using message queues for concurrent users.

7.4 Deployment architecture

7.4.1 Prototype stack

The current implementation runs fully locally using Python modules and a CLI interface:

- LLM orchestrator via Ollama or API (GPT-based).
- Python backend for parsing, normalization, validation, and instance compilation.
- Price forecaster and solver modules imported as separate packages (**implemented** and integrated in the pipeline).

7.4.2 Production-ready deployment

Transitioning to a distributed architecture requires minimal restructuring:

- **Backend API:** wrap the pipeline as REST endpoints (e.g., FastAPI or Flask) exposing routes for conversation, compile, and optimize.
- **Frontend:** simple web dashboard or mobile app built on React/Flutter for non-technical SME users.
- **Persistent storage:** MongoDB or PostgreSQL for storing user sessions, instances, and schedules.
- **Task orchestration:** workers/queues for asynchronous optimization jobs.
- **Monitoring:** dashboards for performance, solver runtime tracking, and data freshness.

7.5 Security and reliability

- **Data privacy:** all user data (machine power, workforce schedules) can remain on-prem if required.
- **Encryption:** HTTPS for API communication and encryption at rest for stored JSON instances.
- **Error recovery:** fallback logic ensures incomplete sessions are saved and resumable.
- **Versioning:** each JSON instance and forecast dataset is versioned for reproducibility.

7.6 Cloud/on-prem integration

Energy-Planner supports two complementary deployment models:

- **On-premise:** suitable for SMEs requiring local control, using lightweight models via Ollama and local solvers (e.g., CBC).
- **Cloud-based:** scalable multi-user deployment with high-performance solvers, live API feeds, and remote forecast updates.
- **Hybrid:** forecasting and optimization in the cloud, user interaction and sensitive data stored locally.

7.7 Future scalability roadmap

- **Voice interaction:** integrate speech-to-text for natural voice-based sessions.
- **Continuous learning:** use aggregated anonymous data to improve forecast accuracy and scheduling heuristics.
- **Multi-user energy pooling:** extend optimization to groups of SMEs sharing distributed generation or storage.
- **Dynamic reporting:** automatic generation of weekly cost and saving summaries via APIs.

7.8 Summary

Energy-Planner's architecture is inherently modular and ready for scaling in both functionality and reach. The transition from prototype to production involves wrapping existing modules into services and connecting to external APIs. Its hybrid deployment

model—local autonomy with optional cloud intelligence—ensures accessibility, security, and future-proof growth for SME energy management.

Chapter 8

User Experience and Roadmap

This chapter outlines the planned evolution of Energy-Planner's user experience (UX) from the current command-line interface (CLI) prototype to a fully interactive, user-friendly digital assistant for SMEs. The roadmap emphasizes accessibility, intuitiveness, and integration with existing business workflows.

8.1 Current UX state

The current prototype operates through a conversational CLI environment (**implemented**):

- Users interact in plain language (Italian or English), describing daily operations and constraints.
- The assistant responds naturally, asking concise questions (1–2 per turn) and confirming each entry before saving.
- Data validation occurs transparently, with warnings and assumptions displayed immediately.
- The process ends with the generation of a validated JSON instance and a readable assumptions report.

This approach ensures transparency and ease of debugging but lacks graphical interactivity and real-time feedback visuals.

8.2 Planned improvements

8.2.1 Short term (prototype to beta)

- **Graphical dashboard:** develop a web-based interface (React or Streamlit) that mirrors the conversational flow but includes dynamic visualizations (Gantt charts, cost heatmaps, peak load curves).

- **Editable summaries:** allow users to modify recognized data (e.g., machine power or activity duration) through simple form widgets before optimization.
- **Instant feedback:** visualize warnings, constraints, and assumed defaults directly in the interface.
- **Scenario export:** one-click download of JSON and schedule results as PDF or CSV.

8.2.2 Medium term (beta to production)

- **Conversational web agent:** migrate the CLI logic into a responsive chat widget connected to the backend API.
- **Voice interaction:** integrate speech recognition for hands-free conversation and text-to-speech for responses.
- **Personalization:** remember company profiles, previous runs, and preferred tariff types to reduce input time.
- **Notification system:** push alerts for forecasted price spikes or weekly optimization reports.

8.2.3 Long term (scaling and integration)

- **Mobile app:** deploy a lightweight Android/iOS version for on-the-go planning and alerts.
- **ERP/IoT integration:** connect directly with industrial systems (e.g., SCADA, smart meters) to retrieve consumption data automatically.
- **Collaboration features:** multi-user support allowing teams (production, energy manager, maintenance) to share and adjust schedules collaboratively.
- **Accessibility:** compliance with WCAG standards, multilingual interface, and high-contrast mode for industrial environments.

8.3 User testing feedback

Early qualitative feedback from SME operators highlighted several strengths:

- The conversational tone reduces the learning curve for non-technical users.
- Default assumptions and explicit warnings increase user trust in recommendations.

- The system's transparency and the clear logging of data were perceived as professional and reliable.

Key areas for improvement include visualization of savings impact and simpler correction of input errors, both addressed in the short-term roadmap.

8.4 UX roadmap summary

Phase	**Focus**	**Deliverables**
Prototype	CLI interaction	Conversation-to-JSON core + validation logs
Beta	Web dashboard + editable UI	Real-time visualization, export tools
Production	Voice + mobile integration	Smart notifications, cloud persistence
Scale-up	ERP/IoT integration	Automated data ingestion, team collaboration

Table 8.1: Planned UX evolution timeline

8.5 Summary

The UX roadmap ensures that Energy-Planner evolves from a research prototype into a practical assistant accessible to any SME operator. The stepwise transition—from command-line chat to visual, conversational, and voice-driven interfaces—aligns with the project's mission: turning complex optimization into simple, actionable dialogue.

Chapter 9

Conclusions

This document presented the complete design and validation of the **Energy-Planner** platform — a conversational decision-support system for small and medium enterprises aiming to optimize electricity usage through predictive intelligence and operations research.

9.1 Achievements

Energy-Planner successfully bridges three traditionally separate domains:

- **Natural language interaction**: users express their operational needs in simple conversational terms, without technical knowledge.
- **Predictive analytics**: hourly price forecasts anticipate cost variations and guide proactive scheduling (**implemented integration**).
- **Optimization modeling**: heuristic and MIP algorithms compute feasible, energy-aware schedules respecting workforce, machine, and power constraints (**both implemented**).

Together, these components enable SMEs to manage energy strategically rather than reactively.

9.2 Key outcomes

The implemented prototype demonstrated that:

- The conversation-to-JSON pipeline converts unstructured information into formal optimization instances reliably and reproducibly.
- The architecture is modular, auditable, and easily extensible to new tariffs, cities, or constraints.

- Heuristic scheduling provides fast, practical performance while maintaining feasibility.
- The system remains operational even with incomplete input, through automatic defaults and assumption logging.
- The user experience can evolve seamlessly from a CLI prototype to a graphical or voice-enabled platform.

9.3 Alignment with Enel Innothon goals

The project meets the evaluation criteria defined in the Enel Innothon challenge:

- **Business value creation:** cost-aware schedules and improved energy management for SMEs.
- **Feasibility:** realistic implementation based on standard Python libraries and scalable architecture.
- **Innovation:** integration of LLM-driven data collection with optimization and forecasting.
- **Scalability:** cloud-ready structure, modular design, and multi-city forecasting support.
- **Code quality:** clean modular repository, reproducible JSON schema, and companion technical documentation.

9.4 Future work

Future iterations will focus on:

- Deploying a fully automated price forecasting API with live updates.
- Integrating advanced hybrid solvers (metaheuristics + MIP warm-starts).
- Extending to multi-day and multi-site optimization.
- Launching a graphical dashboard with dynamic visualizations and voice-based assistant.
- Exploring collaborative optimization between multiple SMEs for shared resources or storage.

9.5 Final remarks

Energy-Planner demonstrates that complex energy optimization can be democratized through intuitive interaction and solid analytical foundations. The system embodies a new generation of digital assistants — capable of understanding human descriptions, predicting costs, and delivering actionable energy-saving recommendations. Its scalable architecture and open design pave the way for real-world deployment, marking a significant step toward intelligent, data-driven energy management for the SME sector.